

System Level Specification Languages

System Level Design and Modeling

II Semester 2025-2026

Preeti Ranjan Panda

Department of Computer Science and Engineering

Indian Institute of Technology Delhi

Why Languages?

- Implementation of conceptual specification models
- Why is natural language not sufficient?
 - ambiguities
- Executable specification
 - machine readable
 - verification, starting point for implementation, documentation

Specification Languages

- System Verilog
- SystemC

System Verilog

- Enhance Verilog with C++ features
 - typedef, struct, pointers, multi-dimensional arrays
- New features
 - assertions
 - dynamic types (parameter type determined at instantiation)
 - data abstraction: interfaces (Object-orientation)
 - including C/C++

Verification: Assertions

```
assert @(posedge clk)
    (! ((Port1mode == READ) && (Port2mode == WRITE)
    && (Port1Addr == Port2Addr)));
```

Assert: Never READ and WRITE from same address

```
assert @(posedge clk)
    (A & !B; A && B; !A &&!B);
```

Assert: Sequence of conditions

System Verilog – Advantages

- Easy to adapt for Verilog HW designer
- New constructs for verification

System Verilog – Disadvantages

- Still need 2 different languages
 - SystemVerilog for HW
 - C/C++ for SW
- Assumes implementation (HW/SW) tradeoffs have already been done

SystemC - Philosophy

- C/C++ already heavily used
 - SW implementations
 - System designers
- Extend the language within the parameters of C/C++
 - add new classes to represent HW features

Guiding Principle: Should be acceptable to Standard C++ Compiler

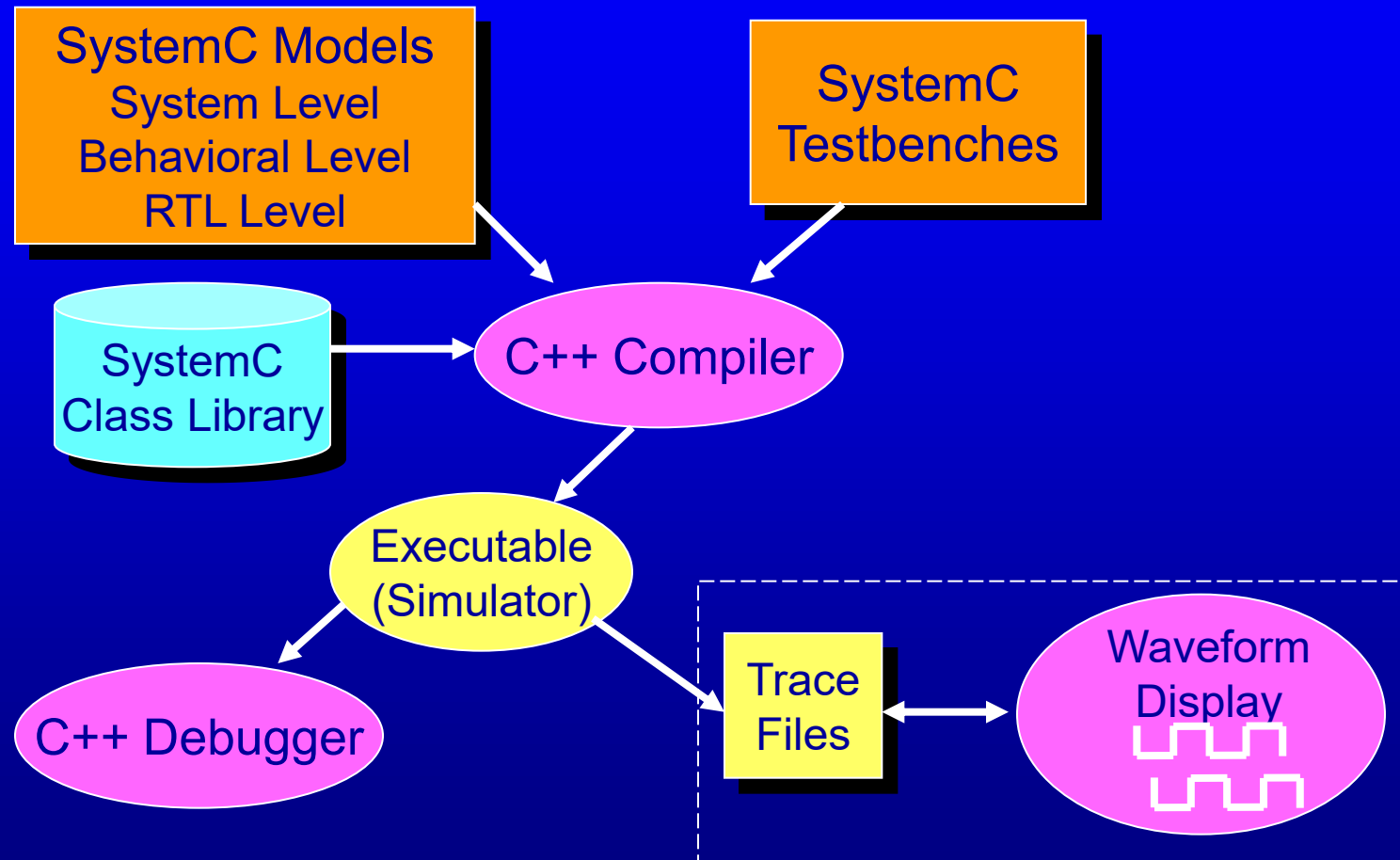
Motivation for C/C++ based design language

- Modeling benefits of C++
 - Abstraction
 - Modularity
 - Object orientation
- Hardware/Software Co-design
- Bridge “specification gap” from System level to RTL
- Efficient exchange of IP

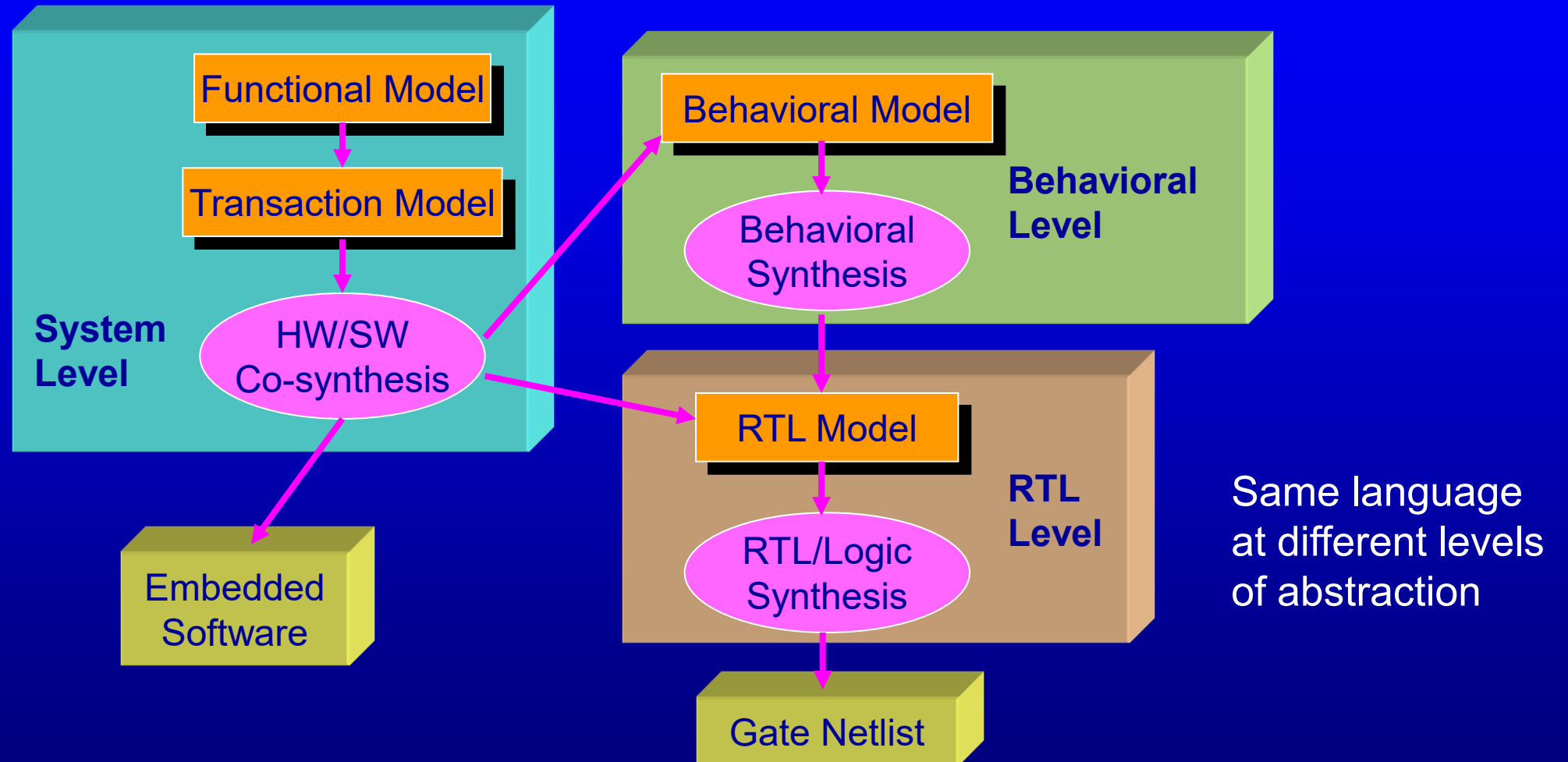
The SystemC Modeling Platform

- Consists of
 - C++ class library
 - Simulation Kernel

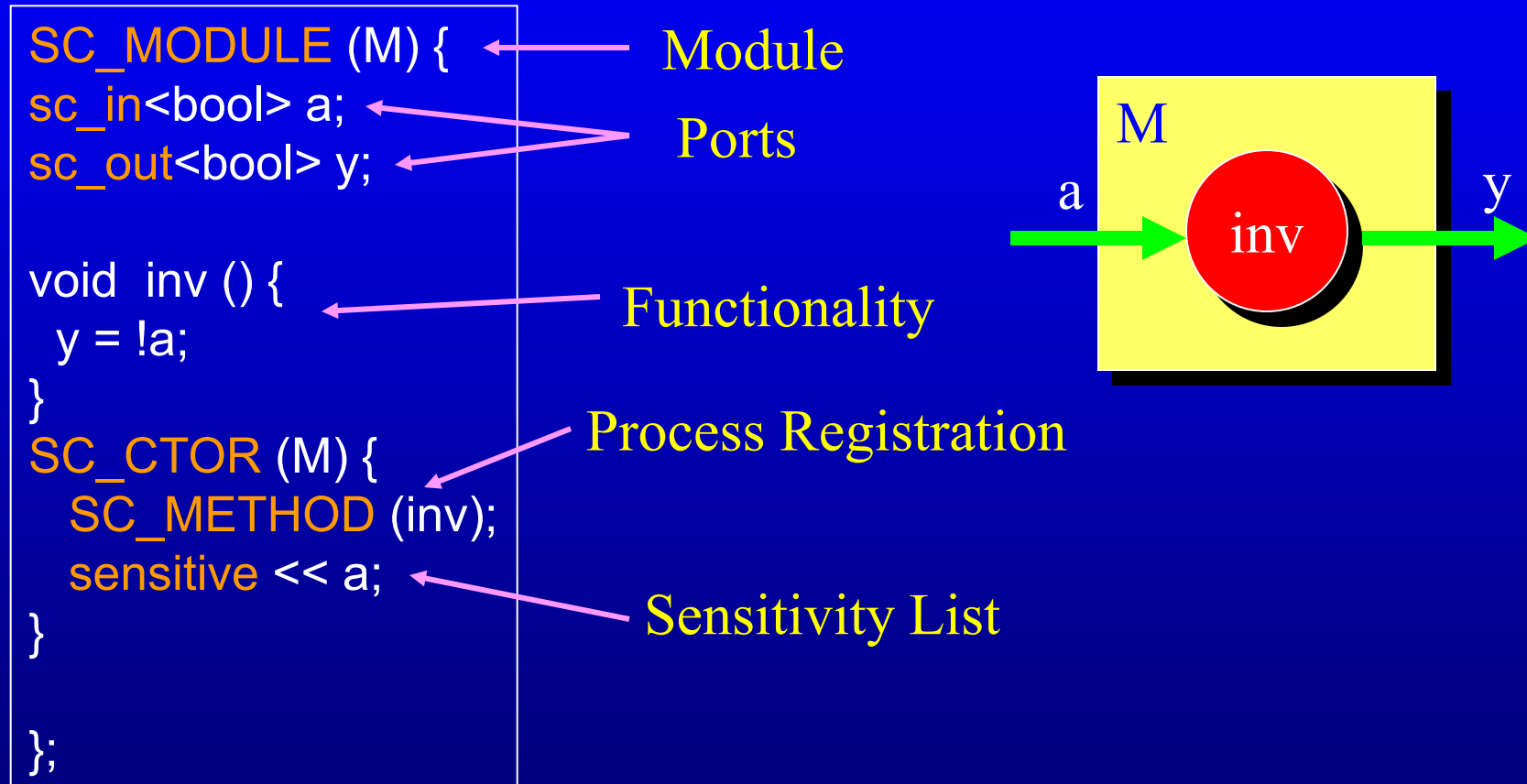
Simulation with SystemC



Implementation and Synthesis



SystemC Details: Modules



Process

- **Used to describe behavior**
 - Basic unit of functionality in SystemC
- **Processes run concurrently**
 - Used to model concurrency in SystemC
 - Code inside a process is sequential
- **A process must be contained in a module**
 - It has to be a function member of module class

Method Process

- Analogous to a C++ function (combinational logic)
- When triggered, executes its body from the beginning to the end
- Needs member variables (data members of module class) to save state
 - FSMs state register – variable cannot be local to process
- **SC_METHOD** registers member function as process

Module & Method Process

```
SC_MODULE(adder) {  
    sc_in<int> input1;           // First input port  
    sc_in<int> input2;           // Second input port  
    sc_out<int> output;          // Output port  
    int result;                  // Internal variable  
    void add_process() {         // Functionality of module  
        result = input1 + input2;  
        output = result;  
    }  
    SC_CTOR(adder) {             // Module Constructor  
        SC_METHOD(add_process);  // Register process  
        sensitive << input1 << input2; // Declare sensitivity  
    }  
};
```

Thread Process

- Analogous to a Unix thread (sequential logic)
- Has the ability to suspend execution by calling `wait()`
- When triggered, continues execution from the point where it was suspended
- Implicitly stores state
 - Local variables can be used to store state
- Use `SC_THREAD` macro to register member function as process

Module & Thread Process

```
SC_MODULE(adder) {  
    sc_in<int> input1;           // First input port  
    sc_in<int> input2;           // Second input port  
    sc_out<int> output;          // Output port  
    int result;                  // Internal variable  
    void add_process() {        // Functionality of module  
        while (true) {  
            result = input1 + input2;  
            output = result;  
            wait();              // Wait until either input1 or input2 changes  
        }  
    }  
    SC_CTOR(adder) {             // Module Constructor  
        SC_THREAD(add_process);  // Register process  
        sensitive << input1 << input2; // Declare sensitivity  
    }  
};
```

SystemC Details:Concurrency

```
SC_MODULE (M) {  
  sc_in<bool> a, b;  
  sc_out<bool> y, z;
```

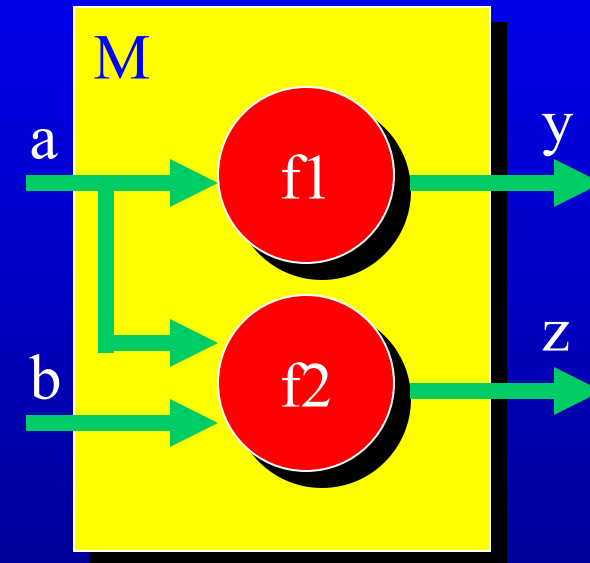
```
  void f1 () {  
    y = !a;  
  }
```

```
  void f2 () {  
    z = a + b;  
  }
```

```
  SC_CTOR (M) {  
    SC_METHOD (f1); ...  
    SC_METHOD (f2);  
  }  
};
```

Functionality

Process Registration



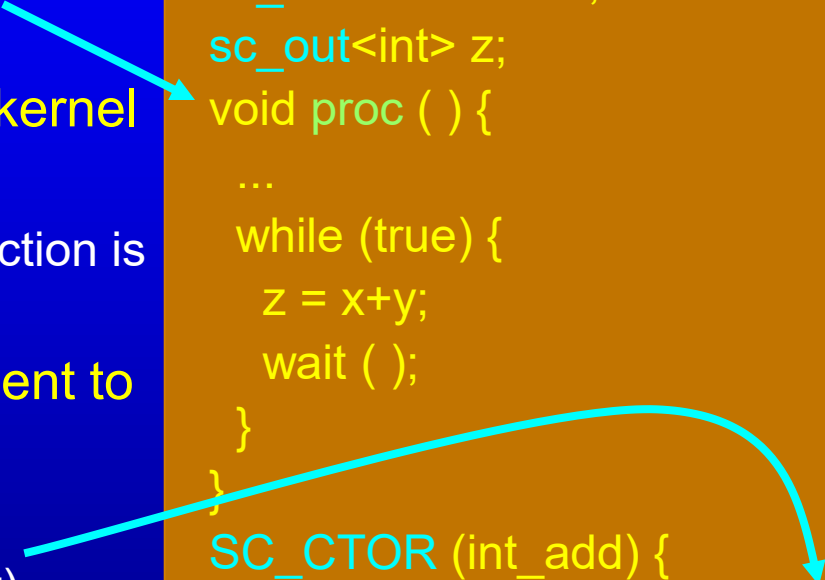
Clocked Thread Process

- Useful for modelling behaviour synchronous with a clock
- Process is triggered on specified clock edge
 - Execution can be suspended by calling `wait()` function
- When triggered, execution is resumed from the point where it was last suspended
- `SC_CTHREAD` registers member function as clocked thread process
- Local variables of function can store state
 - on RESET, values are lost
 - function is called again

Clocked Thread Process

- Clocked thread process function returns nothing (void) and has no parameters
- Function is called ONCE by simulation kernel
 - it may be suspended and restarted
 - it may be RESET, in which case the function is called again
- Sensitivity is indicated in second argument to SC_CTHREAD()
 - clk.pos () : positive edge of clk signal
 - no need for “sensitive <<” (has no effect)

```
SC_MODULE (int_add) {
    sc_in<int> x, y;
    sc_in<bool> clk;
    sc_in<bool> reset;
    sc_out<int> z;
    void proc ( ) {
        ...
        while (true) {
            z = x+y;
            wait ( );
        }
    }
    SC_CTOR (int_add) {
        SC_CTHREAD (proc, clk.pos());
        ...
    }
};
```



Clocked Thread Process

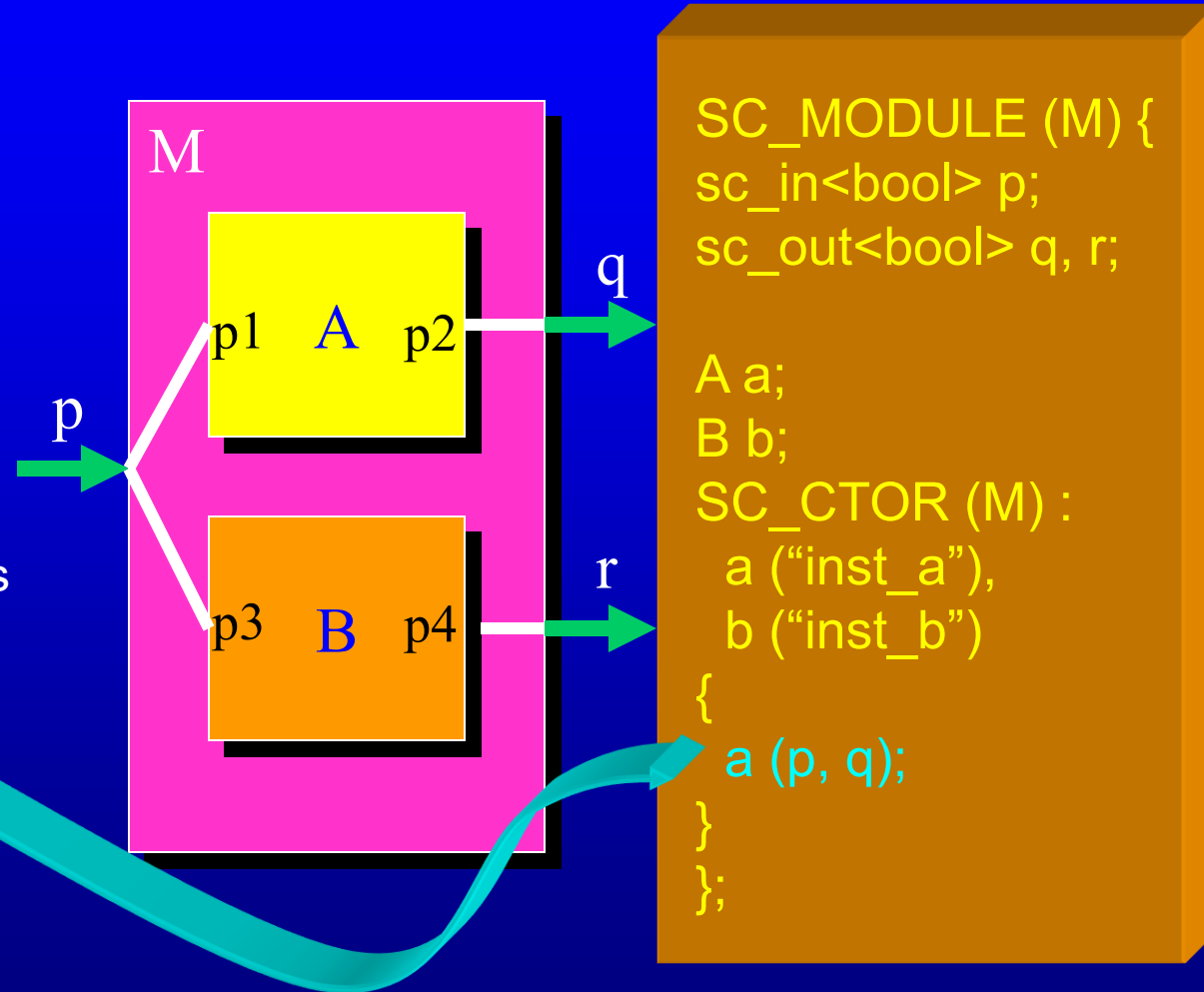
- RESET Behaviour can be specified before the main infinite loop
- `reset_signal_is` specifies the signal and level that cause this behaviour
- Clocked thread is reset when
 - clk event occurs, AND
 - reset condition is TRUE
- If no clock event, then no reset
 - SYNCHRONOUS RESET
- Current process execution is abandoned
- Associated function called again
- Compare with VHDL / Verilog
 - SystemC is cleaner
 - VHDL/Verilog need the reset check on every wait statement

```
SC_MODULE (int_add) {
    sc_in<int> x, y;
    sc_in<bool> clk;
    sc_in<bool> reset;
    sc_out<int> z;
    void proc ( ) {
        if (reset) {
            z = 0; // Reset behaviour
        }
        while (true) {
            z = x+y;
            wait ( );
        }
    }
    SC_CTOR (int_add) {
        SC_CTHREAD (proc, clk.pos());
        reset_signal_is (reset, true);
    }
};
```



Interconnection

- So far, component objects have been created and named
- They need to be interconnected
 - again, in the constructor
 - needs to be done only once
- **Port Mapping**
 - Positional Mapping
 - p is connected to port p1 of a
 - q is connected to port p2 of a
 - order of arguments corresponds to order of port declaration



How Does this Work in C++?

- What C++ construct is invoked in “a (p, q)”?
- This is an overloading of the FUNCTION CALL OPERATOR “()”
 - in sc_module class

```
SC_MODULE (A) {  
    sc_in<bool> p1;  
    sc_out<bool> p2;  
    ...  
};
```

```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> q, r;
```

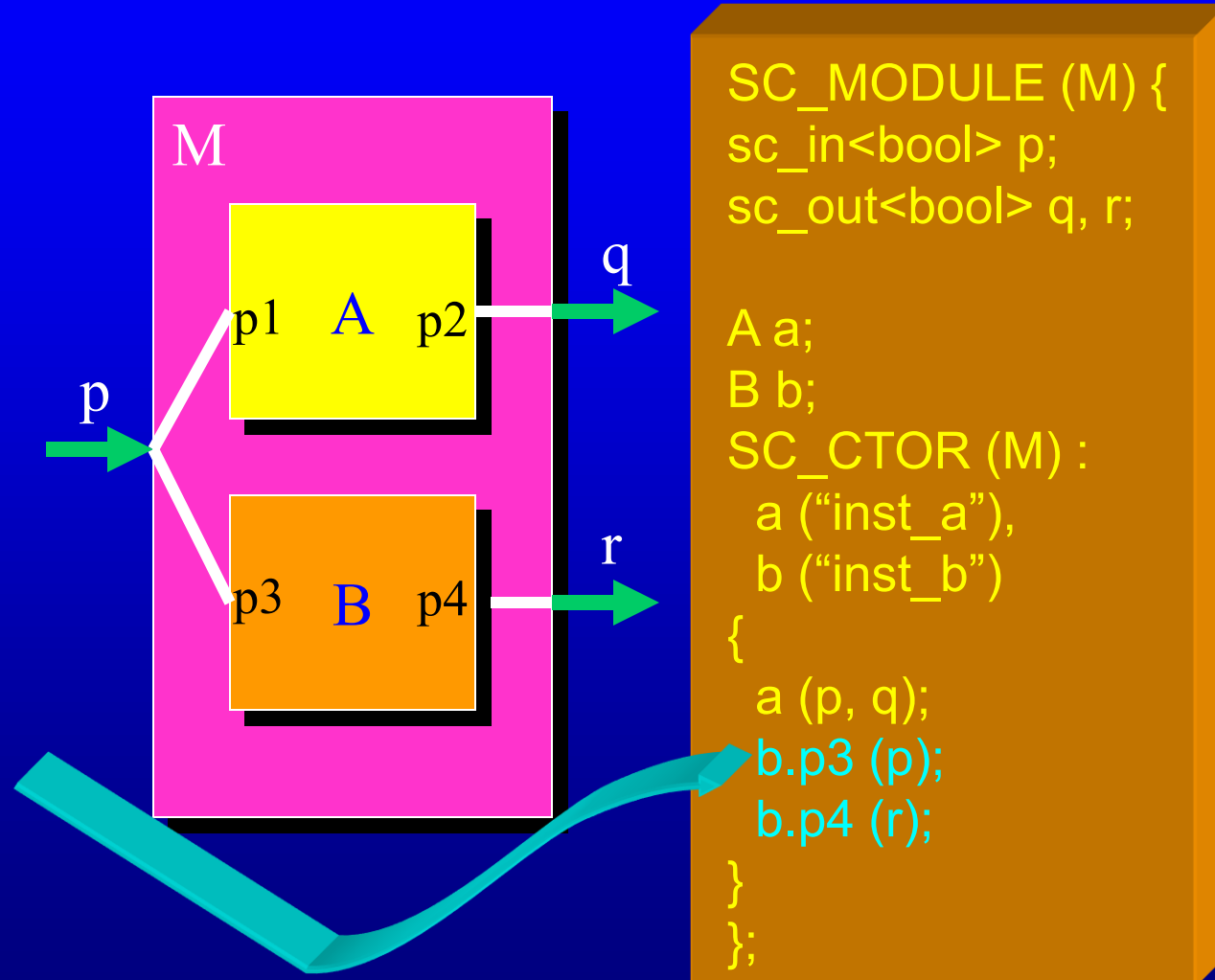
```
    A a;  
    B b;  
    SC_CTOR (M) :  
        a (“inst_a”),  
        b (“inst_b”)  
    {  
        a (p, q);  
    }  
};
```

Two Types of Port Mapping

- **Positional port mapping**
 - mapping identified by position
 - cannot leave any position blank
 - must supply map for all ports
- **Named port mapping**
 - useful (more readable) when there is a large number of ports

Named Port Mapping

- Specify port mappings individually, in separate statements
 - p is connected to port p3 of b
 - r is connected to port p4 of b



How Does this Work in C++?

- What C++ construct is invoked in “b.p3(p)”?
- This is an overloading of the **FUNCTION CALL OPERATOR “()”**
 - in sc_in class

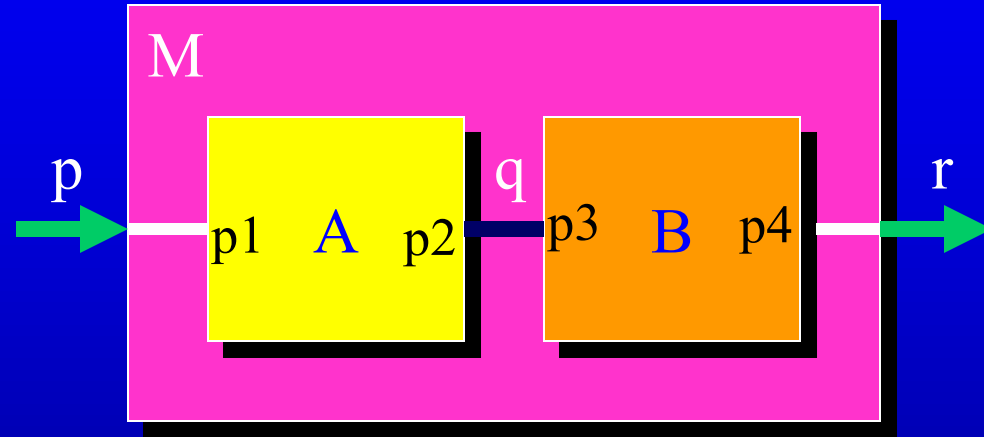
```
SC_MODULE (B) {  
    sc_in<bool> p3;  
    sc_out<bool> p4;  
    ...  
};
```

```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> q, r;
```

```
    A a;  
    B b;  
    SC_CTOR (M) :  
        a (“inst_a”),  
        b (“inst_b”)  
    {  
        a (p, q);  
        b.p3 (p);  
        b.p4 (r);  
    }  
};
```

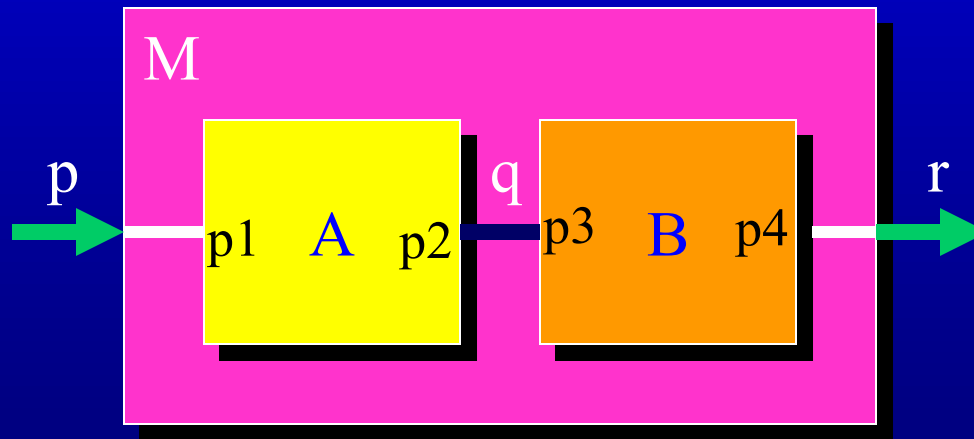
Internal Connectivity with Signals

- p2 and p3 are connected by an internal signal q



Signals are Local to a Module

- use `sc_signals` for internal signals
 - analogous to VHDL signals
 - member data of module class
- signals and ports (`sc_in`, etc.) have the same properties
 - a port is a special type of signal
 - to be generalised later



```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> r;  
  
    sc_signal<bool> q;  
    A a;  
    B b;  
    SC_CTOR (M) :  
        a ("inst_a"), b ("inst_b")  
    {  
        a (p, q);  
        b.p3 (q);  
        b.p4 (r);  
    }  
};
```

Alternative Module Instantiation Technique

- Use pointer declarations
 - data members are not modules, but pointers to modules
- Object instantiation using **new** operator of C++
 - constructors of A,B are called when **new** is called

```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> q, r;
```

```
    A *a;  
    B *b;
```

```
    SC_CTOR (M) {  
        a = new A ("inst_a");  
        ...  
        b = new B ("inst_b");  
        ...  
    }  
};
```

Port Mapping with Pointers

- Since *a* is a pointer, use *(*a)(p,q)* instead of *a(p,q)*
- Since *b* is a pointer, use *b->p3(p)* instead of *b.p3(p)*

```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> q, r;  
  
    A *a;  
    B *b;  
    SC_CTOR (M) {  
        a = new A ("inst_a");  
        (*a) (p, q);  
        b = new B ("inst_b");  
        b->p3 (p);  
        b->p4 (r);  
    }  
};
```

Preferred Instantiation Style?

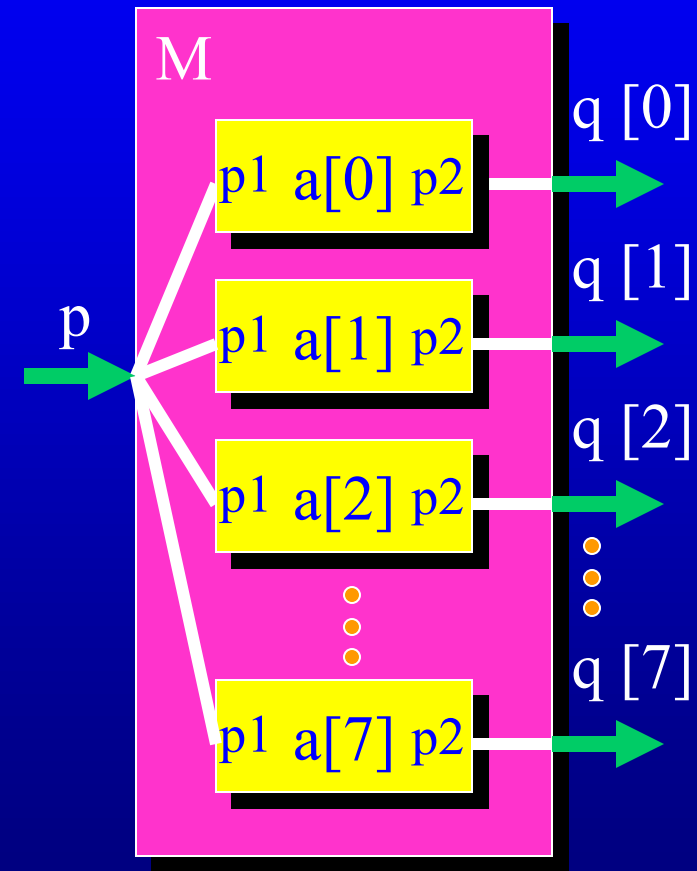
- Both are acceptable
- Non-pointer style looks simpler to those not too comfortable with C/C++
- Pointer style is more general
 - often, more efficient

More Structural Concepts

- Multiple instantiations of the same module
 - use loop construct
- Constructor is just another C++ function, so complex control structures are OK
 - but take care to introduce only necessary one-time statements into constructor
 - any functionality-related statements should be in other member functions, not constructor

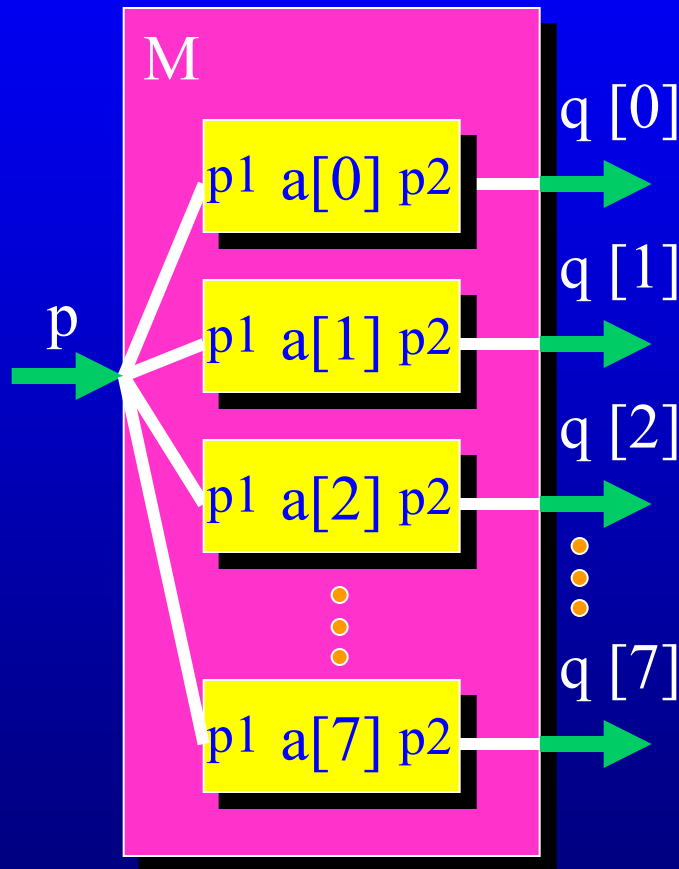
Multiple Instantiations

- We wish to model this structure in SystemC
 - number of instantiations could be large



Looped Instantiation in SystemC

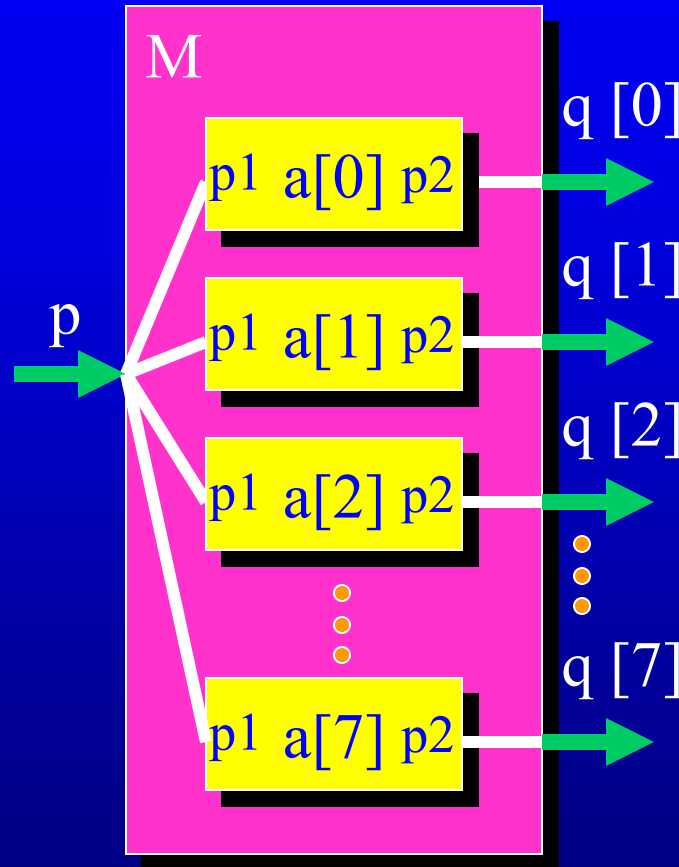
- Declare array of pointers to module
- Place instantiation statement in a loop
- Using pointers is more efficient / elegant



```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> q[8];  
  
    A *a [8];  
    SC_CTOR (M) {  
        int i;  
        for (i = 0; i < 7; i++) {  
            a [i] = new A (...);  
            ...  
        }  
    }  
};
```

Port Maps in Looped Instantiation

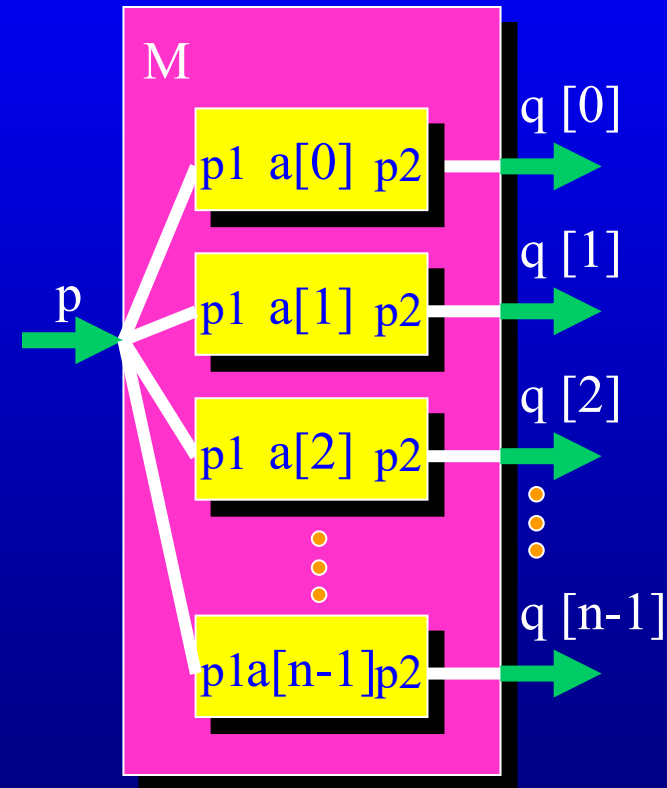
- Each $a[i]$ is a pointer to a module
- $a[i] \rightarrow p1$ refers to port $p1$ of i -th instance
- Positional port map:
 - $a[i] (p, q[i]);$
- Detail omitted:
 - how to compose the constructor argument
 - distinct string needed for each instance



```
SC_MODULE (M) {  
    sc_in<bool> p;  
    sc_out<bool> q[8];  
  
    A *a [8];  
    SC_CTOR (M) {  
        int i;  
        for (i = 0; i < 7; i++) {  
            a [i] = new A (...);  
            a [i]->p1 (p);  
            a [i]->p2 (q [i]);  
        }  
    }  
};
```

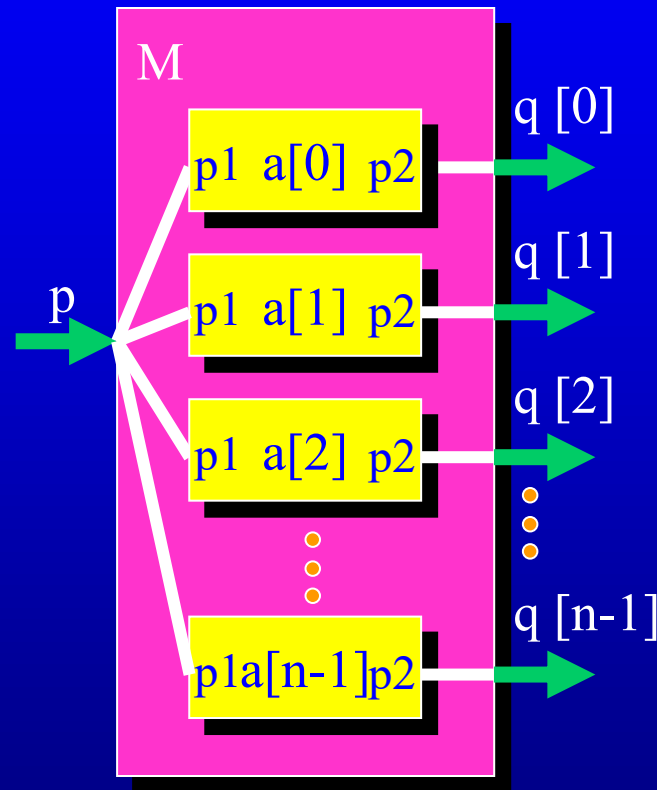
Variable Number of Instantiations

- We wish to model a structure in which there are n instantiations of another module
 - where n is unknown
- E.g., an n -bit ripple carry adder
 - contains n full adders
- Compare: **GENERATE** statement in VHDL



How do we Modify the Previous Model?

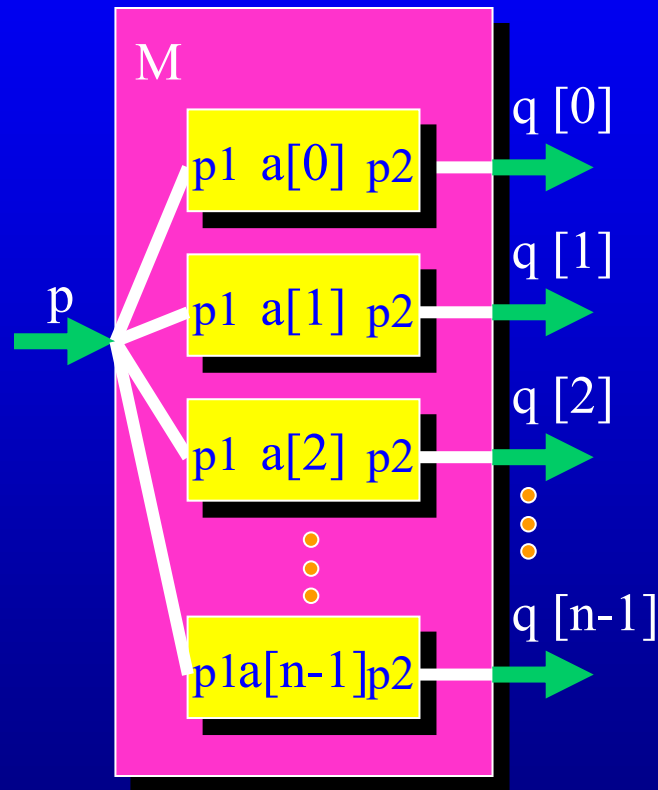
- We need to replace 8 & 7 with parameter n
- ...but C++ arrays should have constant size



```
SC_MODULE (M) {  
  sc_in<bool> p;  
  sc_out<bool> q[8];  
  
  A *a [8];  
  SC_CTOR (M) {  
    int i;  
    for (i = 0; i < 7; i++) {  
      a [i] = new A (...);  
      a [i]->p1 (p);  
      a [i]->p2 (q [i]);  
    }  
  }  
};
```

Solution: Use Templates

- Pass n as template parameter
- This can be specified as array size



```
template <int n>
SC_MODULE (M) {
    sc_in<bool> p;
    sc_out<bool> q[n];

    A *a [n];
    SC_CTOR (M) {
        int i;
        for (i = 0; i < n-1; i++) {
            a [i] = new A (...);
            a [i]->p1 (p);
            a [i]->p2 (q [i]);
        }
    }
};
```

Using the Template Module

- Pass template parameter when instantiating the module
- Ultimately, the array sizes still resolve to compile-time constants
- A class can have any number of template parameters
 - template <int i,int j>...

```
template <int n>
SC_MODULE (M) {
    sc_in<bool> p;
    sc_out<bool> q[n];

    A *a [n];
    SC_CTOR (M) {
        int i;

        for (i = 0; i < n-1; i++) {
            a [i] = new A (...);
            a [i]->p1 (p);
            a [i]->p2 (q [i]);
        }
    }
};
```

```
SC_MODULE (X) {
    ...
    M<4> a;
    M<5> b;
    ...
    SC_CTOR (X) {
        ...
    }
};
```

Specifying Test benches

- Specified with SC_THREAD
 - a test bench is just another process
 - idea similar to other HDLs
- Easily integrated into overall design
- All C++ constructs can be used
 - File I/O, data abstraction, text processing
 - No need for synthesizable subset

Reusing/Refining Test benches

- Test bench can be reused across different steps in the design methodology
- When more detail is added to the design, the test bench may need to be similarly refined